

Uniformed Search: Basic Search Strategies Part-3

Introduction

Imagine trying to find a specific item in a vast, unorganized warehouse, or planning the most efficient route through a complex city. These everyday challenges mirror the fundamental problems faced in Artificial Intelligence: how to systematically explore a space of possibilities to reach a desired outcome. This lecture delves into the foundational search strategies that AI systems employ to navigate such "problem spaces," focusing on methods that operate without specific knowledge about the goal's location—hence, "uninformed" search. Understanding these strategies, their strengths, and their weaknesses is crucial for designing intelligent agents that can effectively solve problems, from simple puzzles to complex decision-making tasks. We will evaluate these algorithms based on critical performance metrics, providing a framework to choose the right tool for the right problem.

Core Concepts

Term	Definition	Significance
Completeness	An algorithm's ability to guarantee finding a path to the goal if one exists from the start state.	Essential for ensuring that a solution is found when possible, even if it's not the best one.
Time Complexity	The computational resources (time) an algorithm requires, often measured by the number of nodes expanded or generated.	Indicates how long an algorithm will take to run, crucial for practical applications.
Space Complexity	The memory resources an algorithm requires, typically measured by the total number of nodes stored in memory.	Determines the memory footprint of an algorithm, vital for resource-constrained environments.
Optimality		Important for applications where the quality of the

Term	Definition	Significance
	An algorithm's ability to always find the least-cost (or shortest) path to the goal.	solution (e.g., efficiency, cost) is paramount.
Systematicity	The property of visiting each search node at most once, avoiding redundant exploration.	Helps prevent infinite loops and improves efficiency by not re-exploring already visited paths.
Branching Factor (b)	The maximum number of actions or successor states possible from any given node in the search tree.	A key parameter influencing the exponential growth of the search space.
Depth of Least-Cost Solution (d)	The depth of the shallowest (or least-cost) goal node in the search tree.	Used for analyzing the performance of algorithms that prioritize shallower solutions.
Maximum Depth (m)	The maximum depth of the search tree, which can potentially be infinite in some problem formulations.	Critical for algorithms that explore deeply, as it can lead to non-termination if infinite.
Frontier	The set of nodes that have been generated but not yet expanded (i.e., their children have not been explored).	Represents the "edge" of the explored search space, central to how search algorithms progress.
Step Cost	The cost associated with moving from one state to an adjacent state (taking an action).	Influences the total path cost and is crucial for determining optimality in non-uniform cost scenarios.

Detailed Analysis

1. Evaluating Search Strategies

To effectively compare and contrast different search algorithms, we use a set of five key characteristics:

- * **Completeness:** Does the algorithm guarantee finding a solution if one exists?
- * **Time Complexity:** How much time does the algorithm take, typically measured by the number of nodes expanded or generated? This is often expressed using parameters b (branching factor), d (depth of the least-cost solution), and m (maximum depth of the search tree).
 - * b : The maximum number of possible actions from any given state. For example, in an N-queens puzzle, b might be N .
 - * d : The depth of the shallowest goal state. This is known for analysis but not by the algorithm itself.
 - * m : The maximum depth of the search tree. This can be infinite if cycles are not handled (e.g., moving a gap up and down repeatedly in an N-puzzle).
- * **Space Complexity:** How much memory does the algorithm require, usually measured by the total number of nodes stored? This also uses b , d , and m .
- * **Optimality:** Does the algorithm always find the least-cost solution? This is particularly relevant when actions have varying costs.
- * **Systematicity:** Does the algorithm visit every search node at most once, avoiding redundant exploration? Non-systematic algorithms might revisit nodes multiple times.

A crucial caveat in search is the issue of **repeated states**. If not handled, a simple graph with cycles can lead to an exponentially growing search tree, even if the underlying state space is small. For instance, if two actions from state A lead to B, and two from B lead to C, a search tree will replicate B twice, C four times, and so on, leading to an exponential explosion.

2. Depth-First Search (DFS)

Depth-First Search is a strategy that explores as far as possible along each branch before backtracking.

- * **Mechanism:**

- * Starts at the root node (e.g., 'a').
- * Explores one child (e.g., 'b') and then recursively explores its children (e.g., 'd') until it reaches a node with no unvisited children or a goal.
- * If it hits a dead end or a non-goal leaf node, it backtracks to the most recent node with unexplored children and continues the search.
- * The process continues until the goal is found or all reachable nodes have been

explored.

- * **Completeness:** DFS is **not complete** if the search tree has infinite depth or contains cycles that are not detected. If the algorithm can get stuck exploring an infinitely deep path, it will never find a goal that exists elsewhere. However, if the search tree is finite, DFS is complete.

- * **Time Complexity:** In the worst case, DFS might explore the entire search tree down to its maximum depth m . Thus, its time complexity is $O(b^m)$, where b is the branching factor and m is the maximum depth. This can be very high if m is large.

- * **Space Complexity:** DFS only needs to store the current path from the root to the current node, along with the unexplored siblings of nodes on that path. In the worst case, this path can be m nodes long, and each node might have b children to keep track of. Therefore, its space complexity is $O(bm)$. This is a significant advantage over Breadth-First Search, as it's linear in depth rather than exponential.

- * **Optimality:** DFS is **not optimal**. It finds the first path it encounters to a goal, which may be a very long or high-cost path, even if a shorter or cheaper one exists.

- * **Video Illustration:** A video demonstration showed DFS starting from a start state and making "random" decisions, exploring deeply into one path. When it hit a dead end, it backtracked. The path it eventually found to the goal was often convoluted and suboptimal, highlighting its non-optimality. The advantage of DFS, as observed, is its lower memory usage.

3. Breadth-First Search (BFS)

Breadth-First Search explores the search space level by level, expanding all nodes at one depth before moving to the next.

- * **Mechanism:**

- * Starts at the root node (e.g., 'a').

- * Expands all its immediate children (depth 1, e.g., 'b', 'c').

- * Then expands all children of depth 1 nodes (depth 2, e.g., 'd', 'e', 'f', 'g'), and so on.

- * It maintains a **frontier** (often a queue) of nodes to be expanded, adding new children to the end of the queue.

- * **Completeness:** BFS is **complete** if the branching factor b is finite. Since it explores all nodes at a given depth before moving deeper, it will eventually find any goal node if one exists.

- * **Time Complexity:** In the worst case, BFS might need to explore all nodes up to the depth d of the shallowest goal. The number of nodes at depth d is b^d . Thus, its time complexity is $O(b^d)$.

- * **Space Complexity:** BFS must store all nodes in the **frontier** at the current depth, as well as all previously generated but unexpanded nodes. In the worst case,

this means storing all nodes at depth d . Therefore, its space complexity is $O(b^d)$. This can be a major drawback for large search spaces.

* **Optimality:** BFS is **optimal** if all step costs are uniform (e.g., each step costs 1). In this scenario, the first goal found will necessarily be the one with the fewest steps, hence the shortest path. If step costs vary, BFS is not guaranteed to be optimal.

* **Video Illustration:** A video showed BFS expanding outwards in all directions from the start state, maintaining a "frontier" (yellow region). It systematically explored all nodes at a given distance before moving further. When all step costs were equal, it found the optimal (shortest) path, demonstrating its optimality in such cases.

4. Uniform Cost Search (UCS)

Uniform Cost Search is a generalization of Breadth-First Search that finds the least-cost path when step costs are not uniform.

* **Mechanism:**

* Expands the node with the lowest cumulative path cost from the start state.

* It uses a priority queue for its **frontier**, ordered by the path cost to each node.

* When a node is expanded, its children are added to the priority queue with their respective path costs.

* **Completeness:** UCS is **complete** if all step costs are positive and non-zero. If zero-cost edges are allowed, it might get stuck in a loop of zero-cost actions, leading to non-termination. If costs can be geometrically decreasing (e.g., 1, 1/2, 1/4), it might take infinite time to reach a certain cost threshold, even if a goal exists.

* **Time Complexity:** The time complexity of UCS is $O(b^{(C^*/\epsilon)})$, where b is the branching factor, C^* is the cost of the optimal solution, and ϵ is the minimum positive step cost. This reflects that it explores nodes up to the cost of the optimal path, divided by the smallest possible step increment.

* **Space Complexity:** Similar to time complexity, its space complexity is $O(b^{(C^*/\epsilon)})$, as it needs to store nodes in its priority queue based on their path costs.

* **Optimality:** UCS is **optimal** even when step costs are non-uniform, provided that step costs are positive. It guarantees finding the path with the lowest total cost.

* **Relation to BFS:** If all step costs are equal (e.g., 1), Uniform Cost Search behaves identically to Breadth-First Search.

Key Takeaways

- [] Search strategies are evaluated based on completeness, time complexity, space complexity, optimality, and systematicity.
- [] b (branching factor), d (depth of least-cost solution), and m (maximum depth) are critical parameters for complexity analysis.

- [] Depth-First Search (DFS) is space-efficient ($O(bm)$) but can be incomplete and non-optimal, with high time complexity ($O(b^m)$).
- [] Breadth-First Search (BFS) is complete and optimal for uniform step costs but can be very memory-intensive ($O(b^d)$) and time-consuming ($O(b^d)$).
- [] Uniform Cost Search (UCS) is complete and optimal for positive, non-zero step costs, even with varying costs.
- [] UCS's complexity is tied to the optimal path cost (C) and minimum step cost (ϵ), making it $O(b^{C/\epsilon})$ for both time and space.
- [] Handling repeated states is crucial to prevent exponential growth of the search tree and ensure systematicity.
- [] The choice of search algorithm depends on the problem's characteristics, such as the likelihood of infinite paths, memory constraints, and the importance of finding the absolute best solution.

Conclusion

The exploration of uninformed search strategies reveals a fundamental trade-off between computational resources and solution quality. Depth-First Search, while memory-efficient, risks getting lost in infinite paths and often yields suboptimal solutions. Breadth-First Search, on the other hand, guarantees completeness and optimality (for uniform costs) but can quickly exhaust memory and time for deep search spaces. Uniform Cost Search extends BFS to handle varying step costs, ensuring optimality in more complex scenarios, but still faces exponential complexity challenges.

These basic strategies form the bedrock of more advanced AI search techniques. Understanding their inherent limitations and strengths is vital for any AI practitioner. For instance, in real-world applications like pathfinding in video games or logistics, the choice between a fast, approximate solution (DFS-like) and a guaranteed optimal one (UCS-like) depends heavily on the specific constraints and requirements of the problem. The next steps in search involve incorporating "informed" knowledge (heuristics) to guide the search more intelligently, moving beyond the brute-force exploration of these uninformed methods.